



Universidad de Guayaquil
Facultad de Ingeniería Industrial
Carrera de Ingeniería en Telemática

Materia:

Ingeniería de Software

Tema:

Etapas 4: Desarrollo del Prototipo Base

Grupo de Trabajo:

Calero Matos Anthony David

Intriago Celi Bladimir Sebastian

Rodriguez Calderon Jordan Josue

Vicuña Aspiazu Mario Samuel

Docente:

Ing. Ángel Plaza Vargas, MSC.

Fecha de Entrega:

16 de junio de 2026

Año Lectivo:

2025 – 2026 CI

Contenido

1.	Introducción.....	5
2.	Objetivos	5
2.1.	Objetivo General	5
2.2.	Objetivos Específicos	5
3.	Descripción de la Etapa.....	5
4.	Implementación del Prototipo Gemini	6
4.1.	Arquitectura Utilizada.....	6
4.2.	Estructura del Proyecto	6
4.3.	Ventana Principal.....	7
4.4.	Sistema de Escenas	8
4.5.	Escenario Base	8
4.6.	Observaciones	9
5.	Implementación del Prototipo Claude	9
5.1.	Arquitectura Utilizada.....	9
5.2.	Estructura del Proyecto	10
5.3.	Ventana Principal.....	13
5.4.	Sistema de Escenas	14
5.5.	Escenario Base	14
5.6.	Observaciones	15
6.	Implementación del Prototipo Deepseek.....	15
6.1.	Arquitectura Utilizada.....	15
6.2.	Estructura del Proyecto	15
6.3.	Ventana Principal.....	17
6.4.	Sistema de Escenas	18
6.5.	Escenario Base	19

6.6. Observaciones	19
7. Pruebas Funcionales	20
7.1. Casos de Prueba	20
7.2. Resultados Obtenidos	20
8. Comparación Técnica	21
8.1. Criterios de Evaluación	21
8.2. Tabla Comparativa	21
8.3. Análisis Técnico	21
9. Resultados Obtenidos	22
10. Discusión	23
11. Conclusiones	23
Anexo	23
Referencias	24

Índice de Figuras

Figura 1. Ventanas Principales Creadas por Gemini	7
Figura 2. Menú principal e interfaz de pantallas	8
Figura 3. Escenario base creado por Gemini	9
Figura 4. Ventanas Principales Creadas por Claude	13
Figura 5. Menú principal y Sistema de Botones Interactivos de Claude.	14
Figura 6. Escenario base creado por Claude	14
Figura 7. Ventanas Principales Creadas por DeepSeek	17
Figura 8. Ventanas de Escenas por DeepSeek	18
Figura 9. Ventanas Principales Creadas por DeepSeek	19

1. Introducción

Luego de finalizar las etapas de planificación, análisis de inteligencias artificiales y modelado del sistema, se procede a la construcción del primer prototipo funcional del videojuego "El Último Bastión".

Esta etapa tiene como finalidad implementar la estructura básica del videojuego siguiendo las arquitecturas propuestas por las inteligencias artificiales Gemini, Claude y DeepSeek. Cada inteligencia artificial desarrollará su propia versión del prototipo respetando la arquitectura planteada durante la Etapa 2.

El desarrollo del prototipo base constituye el primer acercamiento práctico al videojuego y permitirá evaluar la capacidad de las inteligencias artificiales para generar código funcional, organizado y escalable.

2. Objetivos

2.1. Objetivo General

Desarrollar el prototipo base funcional del videojuego "El Último Bastión" utilizando las propuestas arquitectónicas generadas por Gemini, Claude y DeepSeek.

2.2. Objetivos Específicos

- Implementar la ventana principal del videojuego.
- Desarrollar el Game Loop principal.
- Implementar el sistema de escenas.
- Diseñar el escenario base.
- Integrar los componentes fundamentales del juego.
- Comparar las implementaciones generadas por las inteligencias artificiales.

3. Descripción de la Etapa

La presente etapa comprende la construcción del núcleo funcional del videojuego.

Durante esta fase se implementan los elementos mínimos necesarios para ejecutar el juego, incluyendo:

- Inicialización de Pygame.
- Creación de la ventana principal.
- Gestión de eventos.
- Sistema de estados o escenas.
- Menú principal.
- Pantalla de juego.
- Pantalla de pausa.
- Pantalla de derrota.
- Escenario inicial.

4. Implementación del Prototipo Gemini

4.1. Arquitectura Utilizada

Gemini propone una Arquitectura Basada en Componentes y Gestores (Manager Pattern) operando bajo un Game Loop clásico y gobernado por una Máquina de Estados Finitos (FSM). Enfatiza el bajo acoplamiento y el uso de un diseño guiado por datos (Data-Driven) externalizando configuraciones a un archivo JSON.

4.2. Estructura del Proyecto

El_Ultimo_Bastion_V1_Gemini/

```

├── assets/           # Recursos multimedia
|   ├── images/      # Sprites de pixel art/cartoon (torres, enemigos, UI)
|   ├── audio/       # Música de fondo y efectos de sonido (SFX)
|   ├── fonts/       # Fuentes tipográficas
|   └── maps/        # Archivos de datos del mapa (ej. Tiled, JSON)
├── config/          # Archivos de configuración general
└── settings.json    # Resoluciones, volumen, atajos de teclado
    
```

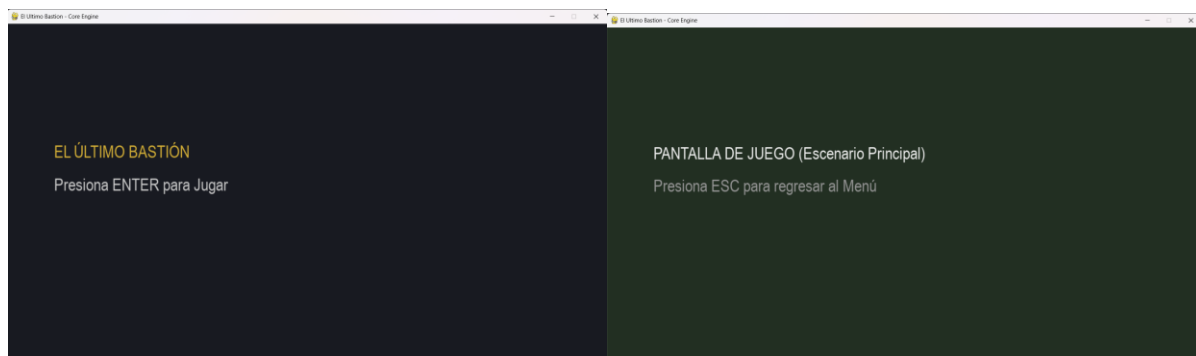
```
| └─ game_data.json    # Estadísticas de enemigos, costos de torres, vida del bastión
| └─ docs/             # Documentación técnica y diagramas UML
| └─ src/              # Código fuente principal
|   └─ core/           # Bucle principal, gestor de estados
|   └─ entities/       # Clases base, enemigos, torres, proyectiles
|   └─ managers/       # Gestores lógicos (oleadas, recursos, construcción)
|   └─ ui/             # Elementos de interfaz gráfica (botones, HUD)
|   └─ utils/          # Funciones auxiliares, matemáticas, colisiones
|   └─ world/          # Lógica del mapa, grilla de construcción, pathfinding
| └─ tests/            # Pruebas unitarias y de integración
| └─ .gitignore        # Archivos a ignorar por Git (ej. __pycache__)
| └─ requirements.txt  # Dependencias del proyecto (ej. pygame-ce==2.3.2)
| └─ main.py           # Punto de entrada de la aplicación
```

4.3. Ventana Principal

Gemini utiliza un archivo central `game_manager.py` que inicializa Pygame y carga la configuración (resolución de 1280x720 y 60 FPS) desde `settings.json`. Emplea un mecanismo de "Doble Búfer" (`pygame.display.flip`) para evitar parpadeos visuales y controla el ciclo del juego calculando el Delta Time.

Figura 1.

Ventanas Principales Creadas por Gemini



Nota. Se observa las ventanas principales del juego desarrollados por Gemini

4.4. Sistema de Escenas

El sistema de escenas se centralizó dentro de `state_manager.py`, donde se programaron las clases `MenuState`, `GameState`, `PauseState` y `GameOverState`. La navegación se realiza por teclado usando índices matemáticos para evitar errores. Para la pausa, no limpia la pantalla completa, sino que dibuja una superficie semitransparente con canal Alpha (SRCALPHA) para lograr un efecto de "overlay".

Figura 2.

Menú principal e interfaz de pantallas



Nota. Menú principal e interfaz de pausa desarrollados por Gemini.

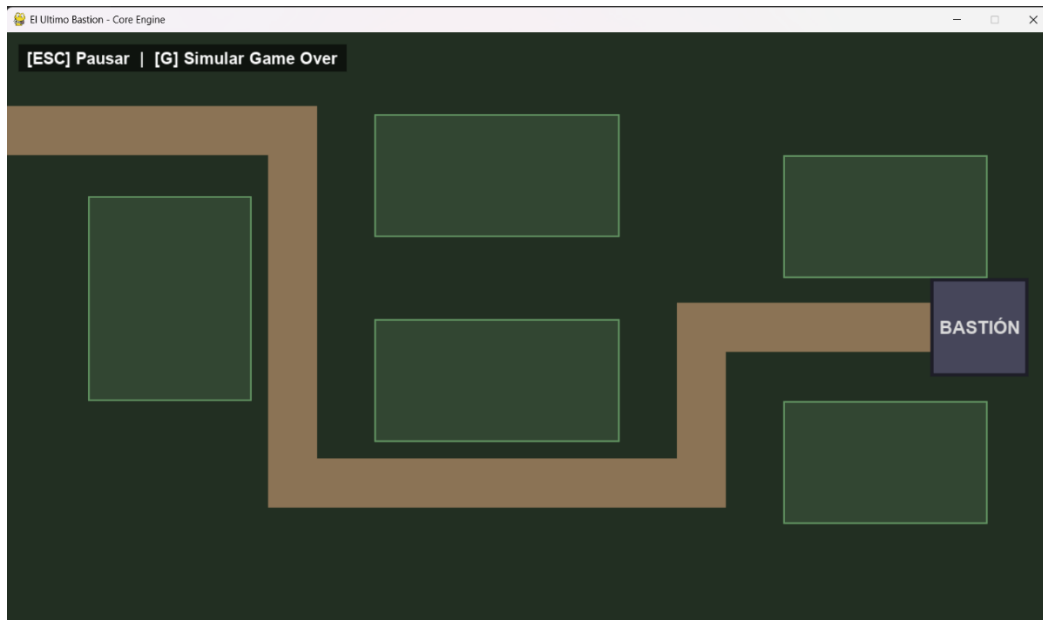
4.5. Escenario Base

Se creó un módulo independiente `world/map.py` encargado de gestionar la geometría del escenario. En lugar de dibujar obstáculos de forma estática, programa la generación

dinámica de los caminos a partir de un arreglo de "waypoints" (nodos de ruta), lo cual asegura precisión para futuras colisiones.

Figura 3.

Escenario base creado por Gemini



Nota. Escenario base implementado por Gemini, mostrando waypoints y zonas construibles.

4.6. Observaciones

Ventajas: Rápida creación de prototipos funcionales y excelente uso de la transparencia (Alpha Blending) para la pausa. Las rutas generadas matemáticamente prevendrán bugs en etapas futuras.

Dificultades: Tendencia a aglomerar código en "God Classes". Centralizó todos los estados del juego en un solo archivo `state_manager.py`, lo que a futuro puede mermar la modularidad y dificultar el mantenimiento.

5. Implementación del Prototipo Claude

5.1. Arquitectura Utilizada

Claude presenta una arquitectura sumamente estructurada en capas combinada con un patrón Observer (Event Bus) innegociable. Separa estrictamente la Capa Lógica (Sistemas) de la Capa de Presentación (Renderizado) para maximizar la mantenibilidad.

5.2. Estructura del Proyecto

El_Ultimo_Bastion_V1_Claude/

```

|
|
| — src/                # Código fuente principal
|
|   | — core/            # Núcleo del motor
|   |
|   |   | — game.py      # Game loop principal, inicialización
|   |   | — state_machine.py  # Máquina de estados (pantallas)
|   |   | — event_bus.py  # Sistema de eventos (Observer)
|   |   | — settings.py   # Constantes globales y config runtime
|   |   | — asset_loader.py # Carga centralizada de recursos
|   |
|   | — entities/        # Entidades del juego
|   |   | — base_entity.py  # Clase base abstracta
|   |   | — enemies/
|   |   |   | — base_enemy.py
|   |   |   | — goblin_raider.py
|   |   |   | — goblin_archer.py
|   |   |   | — goblin_brute.py
|   |   | — defenses/
|   |   |   | — base_defense.py
|   |   |   | — archer_tower.py
|   |   |   | — defensive_wall.py
|   |   | — ballista.py

```

```

| | └─ bastion.py      # El bastión (entidad especial)
| |
| | └─ systems/        # Sistemas de lógica
| |   └─ wave_system.py  # Control de oleadas
| |   └─ combat_system.py # Resolución de combate / colisiones
| |   └─ resource_system.py # Gestión de oro y maná
| |   └─ build_system.py  # Construcción y colocación
| |   └─ pathfinding.py   # Navegación de enemigos (A*)
| |   └─ map_system.py    # Grid del mapa, celdas válidas
| |
| | └─ screens/         # Pantallas (estados)
| |   └─ main_menu.py
| |   └─ gameplay.py
| |   └─ pause_screen.py
| |   └─ defeat_screen.py
| |
| | └─ ui/              # Interfaz de usuario
| |   └─ hud.py          # HUD en gameplay (recursos, vida, oleada)
| |   └─ build_panel.py  # Panel de construcción
| |   └─ tooltip.py      # Tooltips de estructuras
| |   └─ button.py       # Componente botón reutilizable
| |
| | └─ utils/           # Utilidades transversales
| |   └─ timer.py        # Temporizadores reutilizables
| |   └─ animation.py    # Gestión de sprites animados
| |   └─ math_utils.py   # Distancias, vectores 2D

```

```

|
| — assets/                # Recursos estáticos
|   | — sprites/
|   |   | — enemies/
|   |   | — towers/
|   |   | — ui/
|   |   | — map/
|   |   | — effects/
|   | — sounds/
|   |   | — sfx/
|   |   | — music/
|   | — fonts/
|
| — data/                  # Configuración en datos (no código)
|   | — enemies.json      # Stats de todos los enemigos
|   | — towers.json       # Costos y stats de torres
|   | — waves.json        # Definición de cada oleada
|   | — maps/
|   |   | — map_01.json   # Layout del mapa
|
| — tests/                 # Pruebas unitarias
|   | — test_wave_system.py
|   | — test_combat_system.py
|   | — test_resource_system.py
|   | — test_pathfinding.py
|

```

```

|— docs/                # Documentación
| |— architecture.md
| |— game_design.md
| |— api_reference.md
|
|— main.py              # Punto de entrada
|— requirements.txt
|— README.md

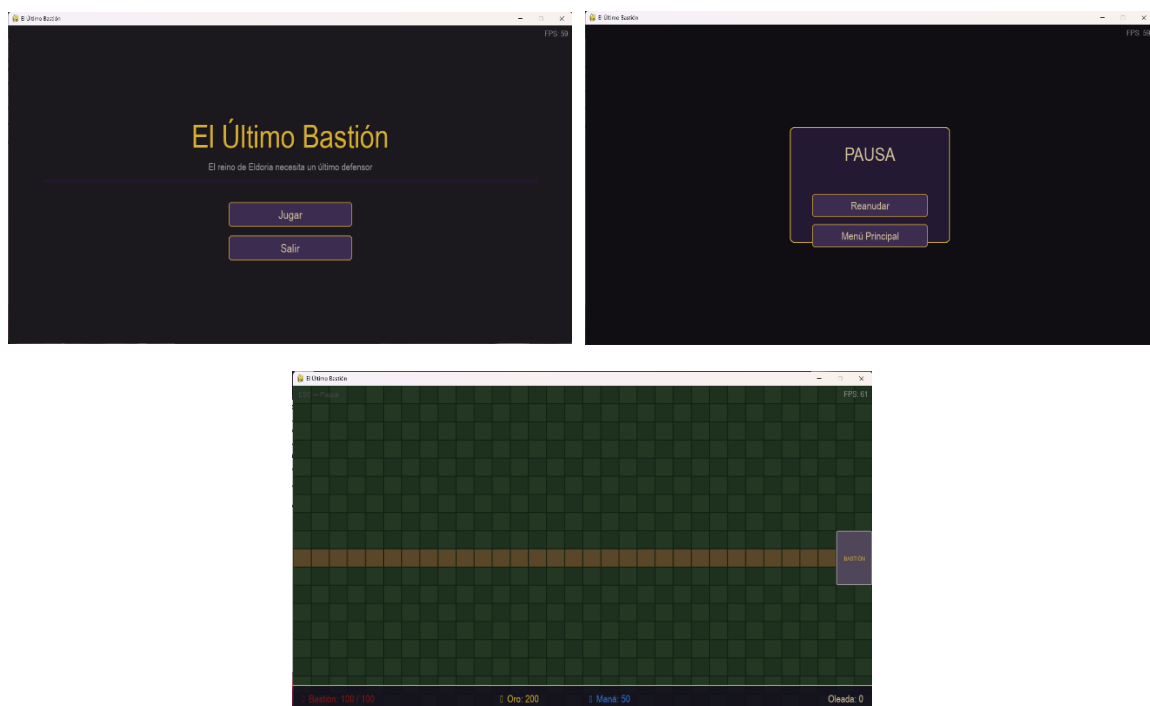
```

5.3. Ventana Principal

Claude configuró el game.py orquestando todo el loop en un orden estricto (eventos → update → clear → draw → flip). Implementó un asset_loader.py con caché centralizada que devuelve "placeholders" (cuadros color magenta) en caso de que falte una imagen para evitar que el juego colapse (crash).

Figura 4.

Ventanas Principales Creadas por Claude



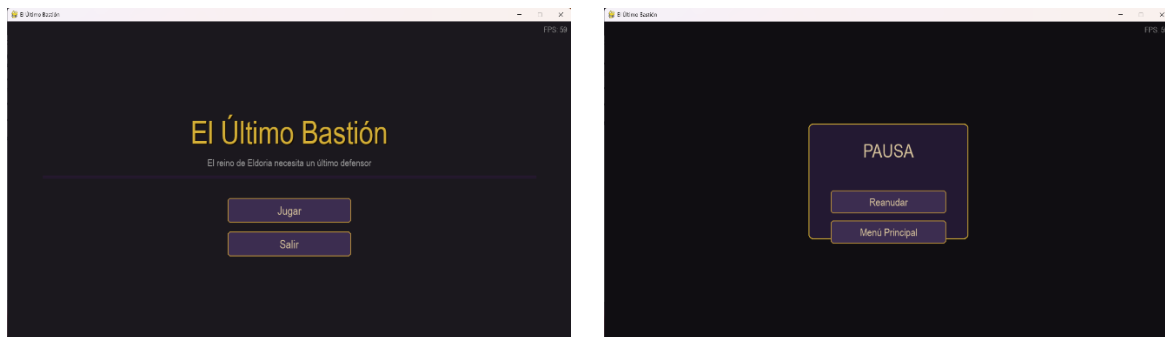
Nota. Ventana principal desarrollada por Claude.

5.4. Sistema de Escenas

El diseño de Claude fue tan completo desde el Prompt 3.1 que generó todas las pantallas en archivos separados desde el inicio (main_menu.py, pause_screen.py, etc.). Utiliza transiciones "lazy" (diferidas al siguiente frame) para evitar corrupción de memoria al cambiar de escena. Implementa total desacoplamiento: las pantallas se comunican publicando eventos como RESTART_GAME en el bus de eventos sin referenciarse directamente.

Figura 5.

Menú principal y Sistema de Botones Interactivos de Claude.



Nota. Sistema de escenarios implementado por Claude.

5.5. Escenario Base

Divide la lógica en dos: MapSystem (solo procesa datos espaciales y colisiones usando un archivo map_01.json) y MapRenderer (solo dibuja píxeles). Además, emplea "Baking" (pre-cálculo) para dibujar el terreno en una superficie fuera de pantalla una sola vez por inicio de partida, ahorrando inmensos recursos de CPU cada frame.

Figura 6.

Escenario base creado por Claude



Nota. Escenario base implementado por Claude con "baking" de texturas.

5.6. Observaciones

Ventajas: Nivel de ingeniería altísimo (Senior). Aplica el Principio de Responsabilidad Única (SRP), incluye un entorno de pruebas unitarias extensivo (32 tests) e implementa técnicas reales de optimización gráfica (Surface baking).

Desventajas: Genera una alta cantidad inicial de archivos (17 en total), lo que requiere una curva de aprendizaje pronunciada para desarrolladores novatos que intenten entender el flujo guiado por eventos.

6. Implementación del Prototipo Deepseek

6.1. Arquitectura Utilizada

DeepSeek apuesta por una estructura de Programación Orientada a Objetos (POO) clásica y muy limpia. Su arquitectura utiliza referencias explícitas y una máquina de estados donde cada estado encapsula su propia lógica de manera aislada.

6.2. Estructura del Proyecto

el-ultimo-bastion/

— main.py	# Punto de entrada, inicializa Pygame y arranca el juego
— config.py	# Constantes globales (resolución, FPS, rutas, balances)

```

├── src/
│   ├── core/
│   │   ├── game.py          # Clase principal Game (bucle, estados)
│   │   └── states.py        # Estados de la máquina (MenuState, GameplayState, etc.)
│   ├── entities/
│   │   ├── enemy.py         # Clase base Enemy y subclases (Saqueador, Arquero, Forzudo)
│   │   ├── defense.py       # Clase base Defense y subclases (TorreArquera, Muro, Balista)
│   │   ├── projectile.py     # Projectiles disparados por defensas
│   │   └── bastion.py        # El bastión (vida, posición)
│   ├── managers/
│   │   ├── wave_manager.py   # Control de oleadas, generación de enemigos
│   │   ├── resource_manager.py # Gestión de oro y maná, validación de gastos
│   │   ├── input_manager.py  # Procesamiento de eventos de teclado y ratón
│   │   └── collision_manager.py # Lógica de colisiones entre proyectiles y enemigos
│   ├── map/
│   │   ├── grid.py          # Cuadrícula para construir defensas
│   │   ├── path.py           # Definición del camino (waypoints)
│   │   └── tile.py           # Representación de cada celda del mapa
│   ├── ui/
│   │   ├── button.py         # Botón genérico para menús
│   │   ├── hud.py            # Interfaz durante el juego (vida, recursos, oleada)
│   │   ├── menu.py           # Pantalla de menú principal
│   │   ├── pause_menu.py     # Pantalla de pausa
│   │   └── game_over.py      # Pantalla de derrota
│   └── utils/
│       ├── asset_loader.py   # Carga de imágenes, sonidos, fuentes

```



```

|   |— timer.py      # Temporizador para cooldowns y oleadas
|   |— vector.py     # Clase Vector2D para cálculos matemáticos
|   |— assets/
|   |— graphics/
|   |   |— enemies/   # Sprites de goblins
|   |   |— defenses/  # Sprites de torres, muros, balista
|   |   |— tiles/     # Imágenes del terreno
|   |   |— ui/        # Botones, fondos de menú
|   |   |— effects/   # proyectiles, explosiones (futuro)
|   |   |— sounds/
|   |   |— music/
|   |   |— sfx/
|   |— fonts/        # Fuentes bitmap
|   |— docs/         # Documentación técnica y de diseño
|   |— tests/        # Pruebas unitarias (p.ej., con pytest)
|   |— data/         # Archivos de definición de niveles, oleadas, balances
|   |— levels/
|   |   |— level_01.json

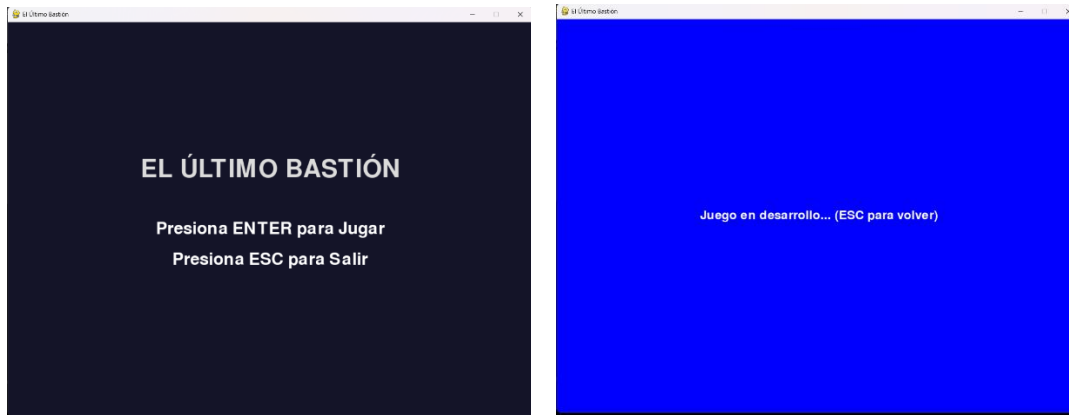
```

6.3. Ventana Principal

El arranque se gestiona en main.py y core/game.py. La configuración global se externaliza puramente en un archivo config.py de Python (1024x768, 60 FPS) lo que evita "números mágicos". El bucle es robusto y delega los eventos dinámicamente al estado activo.

Figura 7.

Ventanas Principales Creadas por DeepSeek



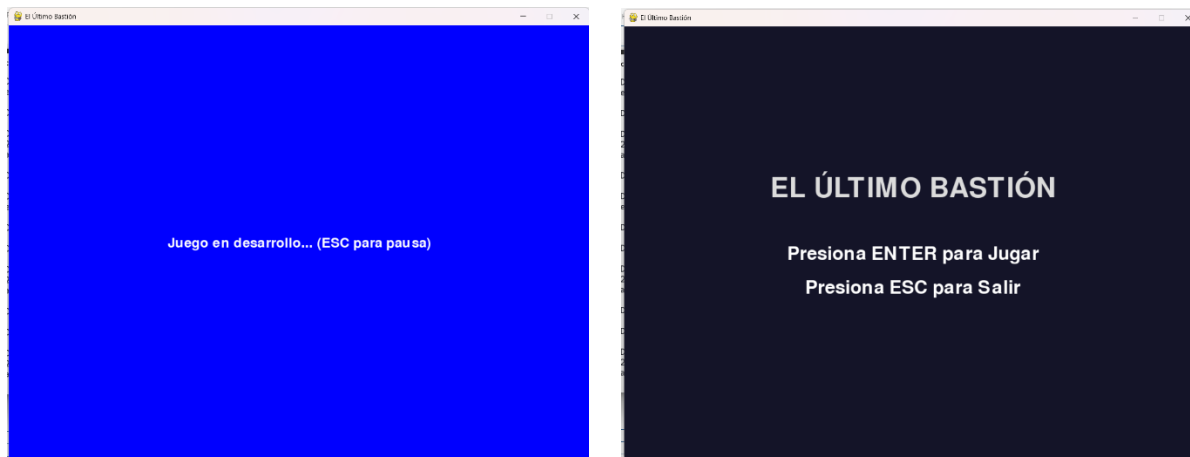
Nota. Ventana principal desarrollada por DeepSeek.

6.4. Sistema de Escenas

La lógica de escenas está agrupada en `states.py`. Al igual que Claude, utiliza herencia basada en una clase abstracta `State`. El cambio de escenas y navegación se activa por teclado. Para la pausa, la clase `PauseState` recibe la instancia activa de `GameplayState` y la congela, superponiendo un overlay semitransparente para indicar el estado pausado sin perder los datos de la partida.

Figura 8.

Ventanas de Escenas por DeepSeek





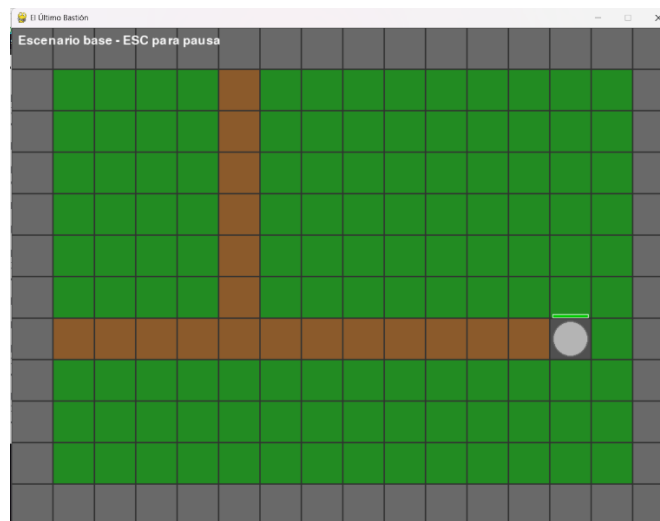
Nota. Pantalla de pausa renderizada por DeepSeek sobre el estado de juego.

6.5. Escenario Base

Aplica el paradigma POO al separar los componentes en `tile.py` (cada celda), `path.py` (los waypoints) y `grid.py` (la cuadrícula). La carga del mapa se basa de momento en una matriz bidimensional codificada directamente en `GameplayState`, extrayendo automáticamente el camino del mismo.

Figura 9.

Ventanas Principales Creadas por DeepSeek



Nota. Escenario base desarrollado por DeepSeek con objetos modulares.

6.6. Observaciones

Ventajas: Un paradigma Orientado a Objetos excelente, sumamente didáctico y fácil de seguir. El acoplamiento por clases como Tile y Grid será muy útil a la hora de verificar colisiones.

Dificultades/Desventajas: Tiende a acoplar lógicamente ciertas clases y utilizó "hardcoding" inicial (matriz manual) para el diseño del nivel en lugar de usar un archivo externo de datos.

7. Pruebas Funcionales

Con el objetivo de verificar el correcto funcionamiento de los prototipos desarrollados, se ejecutaron las siguientes pruebas.

7.1. Casos de Prueba

Código	Descripción	Resultado Esperado
CP-01	Abrir videojuego	Ventana visible
CP-02	Ingresar al menú principal	Menú funcional
CP-03	Iniciar partida	Cambio a Gameplay
CP-04	Pausar juego	Cambio a Pausa
CP-05	Regresar al menú	Retorno correcto
CP-06	Salir del juego	Cierre correcto

7.2. Resultados Obtenidos

Prueba	Gemini	Claude	DeepSeek
CP-01	Exitoso	Exitoso	Exitoso
CP-02	Exitoso	Exitoso	Exitoso
CP-03	Exitoso	Exitoso	Exitoso
CP-04	Exitoso	Exitoso	Exitoso
CP-05	Exitoso	Exitoso	Exitoso
CP-06	Exitoso	Exitoso	Exitoso

Todas las IA lograron superar con éxito los casos de prueba propuestos, entregando ejecutables funcionales libres de "crashes" lógicos durante el flujo de navegación básico.

8. Comparación Técnica

8.1. Criterios de Evaluación

Se establecieron las métricas de Arquitectura, Organización, Modularidad, Escalabilidad, Calidad del código, Claridad, y Facilidad de Mantenimiento. (1 = Muy Deficiente, 5 = Excelente). Los prototipos fueron comparados utilizando los siguientes criterios:

- Organización del proyecto.
- Modularidad.
- Escalabilidad.
- Calidad del código.
- Claridad de explicación.
- Facilidad de implementación.
- Integración de componentes.

8.2. Tabla Comparativa

Criterio	Gemini	Claude	DeepSeek
Organización	4	5	4
Modularidad	4	5	5
Escalabilidad	3	5	4
Calidad del Código	4	5	4
Claridad	4	5	4
Facilidad de Uso	4	5	5
Mantenimiento	3	5	4
Cantidad de Archivos	6	17	10

8.3. Análisis Técnico

Gemini: Ofreció una solución muy rápida orientada a resultados ("Rapid Prototyping").

Aunque su código es sumamente funcional e implementó patrones como el de Gestor (Manager), su modularidad se vio fuertemente afectada al introducir las 4 escenas del juego dentro de un solo archivo (state_manager.py). A largo plazo, esta clase engordará drásticamente perjudicando el mantenimiento.

Claude: Demostró un desempeño técnico sobresaliente. Su enfoque de Ingeniería de Software desacopló totalmente los componentes. Creó interfaces base (BaseScreen), implementó Patrón Singleton y Observer (EventBus), e implementó el principio de Responsabilidad Única aislando la lógica del mapa (MapSystem) de su renderizado (MapRenderer). Sumado a ello, generó una suite completa de pruebas unitarias (test_event_bus.py, test_map_system.py) garantizando robustez total.

DeepSeek: Aportó el diseño Orientado a Objetos más comprensible. Las responsabilidades se dividen en entidades lógicas (Grid, Tile, Path) muy intuitivas. No obstante, falló ligeramente en el desacoplamiento estricto al mezclar los datos crudos del mapa dentro de la misma lógica del estado (GameplayState), mientras que sus competidores utilizaron JSON.

9. Resultados Obtenidos

Durante la presente etapa se logró:

- Implementar la ventana principal y el Game Loop controlando el Delta Time para "El Último Bastión".
- Crear el sistema básico de escenas (Menu, Gameplay, Pause, GameOver).
- Diseñar y renderizar el escenario inicial, demostrando caminos de enemigos y zonas de construcción.
- Validar funcionalmente las tres versiones independientes del prototipo.
- Generar un análisis técnico sobre el impacto a futuro del código brindado por cada IA.

10. Discusión

El desarrollo de la Etapa 4 puso a prueba las propuestas teóricas de la Etapa 2 y 3, revelando que si bien el diseño abstracto puede ser similar, la ejecución algorítmica de cada IA varía considerablemente en su madurez técnica.

Claude se perfila como un "Arquitecto Senior" preocupado por el futuro del proyecto (uso de Event Bus, pre-renderizado de superficies y pruebas automáticas), mientras que Gemini asume un rol de "Desarrollador Junior/Mid", enfocado en dar resultados rápidos y visuales, pero descuidando la fragmentación de responsabilidades.

DeepSeek sirve como un puente altamente didáctico y ordenado, estructurando de manera impecable sus objetos y explicaciones, ideal para contextos académicos.

11. Conclusiones

Las tres inteligencias artificiales demostraron una capacidad plena para construir la base fundacional y funcional de un videojuego desde cero utilizando Pygame. La verdadera diferencia entre ellas radica en cómo aseguran el mantenimiento del código. Claude es el indiscutible ganador en términos de escalabilidad y rigor técnico, previniendo escenarios de optimización y colapso desde las primeras líneas y finalmente la sólida infraestructura lograda por los modelos (especialmente el aislamiento de EventBus y separación por States) confirma que el proyecto está sólidamente preparado para soportar entidades dinámicas e interacciones complejas como la generación de torres y oleadas de enemigos en la próxima fase.

Anexo

Link de Recursos y Proyecto

https://github.com/mariovicunaa/Proyecto_El_Ultimo_Bastion

Referencias

- Mejía Trejo, J. (2024). *Principios de aseguramiento de calidad para el diseño de software: innovación de procesos en las tecnologías de información*: (1 ed.). Academia Mexicana de Investigación y Docencia en Innovación (AMIDI).
<https://elibro.net/es/lc/uguayaquil/titulos/250101>
- Schwaber, K., & Sutherland, J. (2020, 11). *La Guía Scrum*. Scrumguides. Retrieved 06 11, 2026, from <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf>
- Winters, T. & Manshreck, T. (2022). *Ingeniería de software en Google: Lecciones sobre programación aprendidas a lo largo del tiempo*: (1 ed.). Marcombo.
<https://elibro.net/es/lc/uguayaquil/titulos/281863>